

• JAVA OOP •

OOP Tutorial 3

Project

Object-Oriented Programming in Java

10 Core Concepts · IntelliJ IDEA 2026.1.1

01 StudentDetails.java

Encapsulation — Student Profile with Getters & Setters

The screenshot displays an IDE interface with the following components:

- Open Files:** A list of open files including `StudentDetails.java`, `Employee.java`, `Courses.java`, `Fee.java`, and `Grade.java`.
- Structure:** A tree view showing the project structure. The `StudentDetails` class is expanded, showing fields: `name: String`, `rollNumber: int`, and `faculty: String`. It also lists methods: `StudentDetails(String, int, String)` and `getName(): String`.
- Code Editor:** The `StudentDetails.java` file is open, showing the following code:

```
1 class StudentDetails {
34
35 > public void setFaculty(String faculty) { this.faculty = faculty; }
38
39 // Display Student Profile
40 > public void displayProfile() { 1 usage
41     System.out.println("----Student Profile----");
42     System.out.println("Name: " + name);
43     System.out.println("Roll Number: " + rollNumber);
44     System.out.println("Faculty: " + faculty);
45 }
46
47 // Main Method
48 public static void main(String[] args) {
49     StudentDetails student = new StudentDetails ( name: "Ram", rollNumber: 95, f
50     //Display student profile
```
- Run:** The `StudentDetails` class is selected for execution. The output console shows:

```
----Student Profile----
Name: Ram
Roll Number: 95
Faculty: BCSIT

Process finished with exit code 0
```
- Bottom Bar:** The breadcrumb navigation shows the path: `Hello World > src > StudentDetails > main`. A watermark for "Activate Windows" is visible in the bottom right corner.

02 Courses.java

Object Arrays — Student Course Registration

The screenshot displays an IDE interface with the following components:

- Open Files:** A list of files in the 'Hello World' project, including 'src', 'AgePredictor', 'AttendancePercentage.java', 'BasicCalculator', 'Borrowing.java', and 'Courses.java'.
- Structure:** A tree view showing the class hierarchy: 'Course' (with attributes 'name: String' and methods 'Course(String)', 'Course()') and 'StudentName' (with attributes 'name: String' and 'courses: Course[]').
- Code Editor:** The 'Courses.java' file is open, showing the following code:

```
8 class StudentName{ 2 usages
15 void displayInfo(){ 1 usage
18     for (Course c: courses){
19         System.out.println(c.name);
20     }
21 }
22 }
23
24 }
25
26 public class Courses{
27     public static void main(String[] args){
28         Course c1 = new Course( name: "java");
29         Course c2 = new Course( name: "Database");
30         Course[] courses = {c1,c2};
31         StudentName studentname = new StudentName( name: "Ram",courses);
32         studentname.displayInfo();
33     }
34 }
```
- Run:** The 'Courses' class is selected for execution. The output shows:

```
Student: Ram
Registered Courses:
java
Database
Process finished with exit code 0
```
- Bottom Bar:** A path indicator shows the current location: 'Hello World > src > Courses.java > Courses > main'.

03 Employee.java

Inheritance — AdminStaff & Teacher Extending EmployeeDetails

The screenshot shows an IDE with the following components:

- Open Files:** A list of files in the project, including `AgePredictor`, `AttendancePercentage.java`, `BasicCalculator`, `Borrowing.java`, and `Courses.java`.
- Structure:** A tree view showing the class hierarchy: `EmployeeDetails` (with fields `name: String` and `id: int`, and methods `EmployeeDetails(String, int)` and `displayDetails(): void`) and `Teacher`.
- Code Editor:** The `Employee.java` file is open, showing the following code:

```
34 class AdminStaff extends EmployeeDetails { 2 usages
37     public AdminStaff(String name, int id, String Department) { 1 usage
38         super(name, id);
39         this.Department = Department;
40     }
41
42     @Override 2 usages
43     public void displayDetails() {
44         System.out.println("AdminStaff Name: " + name);
45         System.out.println("AdminStaff id: " + id);
46         System.out.println("Department: " + Department);
47     }
48 }
49 //Main class
50 public class Employee {
51     public static void main(String[] args) {
```
- Run Console:** The output of the program is displayed, showing the details of a `Teacher` and an `AdminStaff` object:

```
Teacher id: 45
Subject: Programming
AdminStaff Details:
AdminStaff Name: sumi
AdminStaff id: 30
Department: Administration
```
- Windows Taskbar:** The bottom of the screen shows the Windows taskbar with several open applications, including a web browser and a file explorer.

04 Fee.java

Abstract Class & Polymorphism — Undergraduate & Graduate Fee Calculation

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar includes icons for file operations, running, debugging, and search, along with a 'Unlock Ultimate' button. The 'Open Files' panel on the left shows the project structure: 'Hello World' (C:\Users\user\IdeaProjects\Hello World) with a 'src' folder containing 'AgePredictor', 'AttendancePercentage.java', 'BasicCalculator', 'Borrowing.java', and 'Courses.java'. The 'Structure' panel below it shows the 'StudentData' class with attributes 'name: String' and 'creditHours: int', and methods 'StudentData(String, int)', 'calculateFee(): double', and 'displayInfo(): void'. The main editor shows the 'Fee.java' file with the following code:

```
23 class Graduate extends StudentData { 1 usage
24     Graduate(String name, int creditHours) { super(name, creditHours); }
27     @Override 1 usage
28     public double calculateFee() {
29         return creditHours * 400.00 + 500.00; // $400/credit + $500 research fee
30     }
31 }
32
33 public class Fee{
34     public static void main(String[] args){
35         StudentData[] students = {new Undergraduate( name: "Aman", creditHours: 15),
36             new Graduate( name: "Aney", creditHours: 9)};
37         for (StudentData s : students)
38             s.displayInfo();
39     }
40 }
```

The 'Run' panel at the bottom shows the execution of the 'Fee' class. The command used is: `"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=49848" -Dfile`. The output is:

```
Name: AmanFee: $3000.0
Name: AneyFee: $4100.0
Process finished with exit code 0
```

The status bar at the bottom indicates the current file path: 'Hello World > src > Fee.java > Fee > main'.

05 Grade.java

Interface — Engineering & Management Grade Calculation

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar shows various icons for file operations and development tools. The 'Open Files' panel on the left lists the project structure, including 'Hello World' and its subdirectory 'src'. The 'Structure' panel shows the class hierarchy, with 'ResultProcessor' as the base interface, and 'EngineeringDepartment' and 'ManagementDepartment' as implementing classes. The main editor window shows the 'Grade.java' file, which implements the 'ResultProcessor' interface. The code defines a 'ManagementDepartment' class that implements the 'calculateGrade' method, and a 'Grade' class with a 'main' method that creates instances of 'EngineeringDepartment' and 'ManagementDepartment' and calls their 'calculateGrade' methods with marks of 65 and 55 respectively. The 'Run' panel at the bottom shows the execution output, which displays 'Engineering Grade: B' and 'Management Grade: C', followed by 'Process finished with exit code 0'. The status bar at the bottom indicates the current file path as 'Hello World > src > Grade.java > Grade > main'.

```
18 class ManagementDepartment implements ResultProcessor{ 1 usage
20     public void calculateGrade(int marks) {
25         else if (marks >= 45)
26             System.out.println("Management Grade: C");
27         else
28             System.out.println("Management Grade: D");
29     }
30 }
31 public class Grade{
32     public static void main(String[] args){
33         ResultProcessor eng = new EngineeringDepartment();
34         ResultProcessor mgmt = new ManagementDepartment();
35         eng.calculateGrade( marks: 65);
36         mgmt.calculateGrade( marks: 55);
37     }
38 }
```

Run: Grade

```
"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=49844" -Dfile
Engineering Grade: B
Management Grade: C

Process finished with exit code 0
```

Activate Windows

Go to World > src > Grade.java > Grade > main

06 Borrowing.java

Polymorphism — Student & Teacher Borrow Limits

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar includes icons for file operations and a search icon. The 'Open Files' panel on the left shows the project structure: 'Hello World' (C:\Users\user\IdeaProjects\Hello World) with a 'src' folder containing 'AgePredictor', 'AttendancePercentage.java', 'BasicCalculator', 'Borrowing.java', and 'Courses.java'. The 'Structure' panel below it shows the class hierarchy: 'Member' (with 'getBorrowLimit(): int'), 'StudentMember' (with 'getBorrowLimit(): int'), and 'TeacherMember' (with 'getBorrowLimit(): int'). The main editor window shows the 'Borrowing.java' file with the following code:

```
7      class StudentMember extends Member { 1 usage
8
9      @Override 2 usages
10         public int getBorrowLimit() { return 5; }
11     }
12
13     class TeacherMember extends Member { 1 usage
14
15     @Override 2 usages
16         public int getBorrowLimit() { return 10; }
17     }
18
19     public class Borrowing {
20
21     public static void main(String[] args){
22         Member s = new StudentMember();
23         Member t = new TeacherMember();
24         System.out.println("Student Limit: " + s.getBorrowLimit());
25         System.out.println("Teacher Limit: " + t.getBorrowLimit());
26     }
```

The 'Run' panel at the bottom shows the execution of 'Borrowing'. The command used is: `"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=49856" -Dfile.encoding=UTF-8`. The output is: `Student Limit: 5` and `Teacher Limit: 10`. The process finished with exit code 0. A watermark 'Activate Windows' is visible in the bottom right corner.

07 AttendancePercentage.java

Inheritance & Method Overriding — Engineering & Medical Attendance

The screenshot shows an IDE with the following components:

- Open Files:** A list of files including `AgePredictor`, `AttendancePercentage.java`, `BasicCalculator`, `Borrowing.java`, and `Courses.java`.
- Structure:** A tree view showing the class hierarchy: `Attendance` (with `calculateAttendance(int, int): double`), `EngineeringAttendance` (with `calculateAttendance(int, int): double`), and `MedicalAttendance` (with `calculateAttendance(int, int): double`).
- Code Editor:** Displays the `AttendancePercentage.java` file. The code includes:

```
class EngineeringAttendance extends Attendance {  
    // ...  
}  
  
class MedicalAttendance extends Attendance {  
    @Override  
    public double calculateAttendance(int attended, int total) {  
        return (attended * 100.00) / total;  
    }  
}  
  
public class AttendancePercentage {  
    public static void main(String[] args) {  
        // ...  
    }  
}
```
- Run:** A dropdown menu showing the execution of `AttendancePercentage`. The output is:

```
"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:..." -Dfile  
Engineering: 80.0%  
Medical: 90.0%  
Process finished with exit code 0
```
- Code Completion Popup:** A popup menu is visible over the `main` method, showing the following options:
 - Modifier 'public' is redundant for 'main' method on Java 26
 - Remove 'public' modifier (Alt+Shift+Enter)
 - More actions... (Alt+Enter)
 - AttendancePercentage (selected)
 - public static void main(String[] args)
 - Hello World

08 Notifications.java

Interface Polymorphism — Email & SMS Notification System

The screenshot displays the IntelliJ IDEA IDE interface. On the left, the 'Open Files' pane shows the project structure with 'Hello World' as the root, containing a 'src' folder with several Java files. The 'Structure' pane shows the class hierarchy: 'Notification' (with 'sendNotification(String): void'), 'EmailNotification' (with 'sendNotification(String): void'), and 'SMSNotification' (with 'sendNotification(String): void'). The main editor shows the 'Notifications.java' file, which defines an interface 'Notification' and two implementing classes, 'EmailNotification' and 'SMSNotification'. The 'main' method in the 'Notifications' class creates instances of these classes and calls their 'sendNotification' methods. The 'Run' pane at the bottom shows the execution output: 'Email Sent: Exam Schedule Released' and 'SMS Sent: Fee Payment Reminder', followed by 'Process finished with exit code 0'. The status bar at the bottom indicates the current file is 'Notifications.java' in the 'src' folder of the 'Hello World' project.

```
4 class EmailNotification implements Notification { 1 usage
8     }
9 }
10 class SMSNotification implements Notification { 1 usage
11     @Override 2 usages
12     public void sendNotification(String message){
13         System.out.println("SMS Sent: " + message);
14     }
15 }
16 public class Notifications{
17     public static void main(String[] args){
18         Notification email = new EmailNotification();
19         Notification sms = new SMSNotification();
20         email.sendNotification("Exam Schedule Released");
21         sms.sendNotification("Fee Payment Reminder");
22     }
}
```

Run Notifications x

"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=49840" -Dfile

Email Sent: Exam Schedule Released

SMS Sent: Fee Payment Reminder

Process finished with exit code 0

Activate Windows

Hello World > src > Notifications.java

09 UniqueID.java

Static Fields — Auto-Generated Student IDs

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar shows 'Open Files' and a list of open files: Fee.java, Grade.java, Notifications.java, UniqueID.java (active), and Eligibility.java. The left sidebar shows the project structure for 'Hello World' at 'C:\Users\user\IdeaProjects\Hello World', with a 'src' folder containing files like AgePredictor, AttendancePercentage.java, BasicCalculator, Borrowing.java, and Courses.java. Below the project structure is a 'Structure' view showing the 'Student' class with fields 'counter: int = 1000' and 'studentID: int', and methods 'Student()' and 'display(): void'. The 'UniqueID' class is also listed. The main editor shows the code for 'UniqueID.java' with the following content:

```
1  class Student { 4 usages
6      }
7      public void display() { 2 usages
8          System.out.println("Student ID: " + studentID);
9      }
10 }
11 public class UniqueID {
12     public static void main (String[] args){
13         Student s1 = new Student();
14         Student s2 = new Student();
15         s1.display();
16         s2.display();
17     }
18 }
19 }
20 }
```

Below the code editor is a 'Run' button and a 'UniqueID' tab. The bottom panel shows the execution output:

```
"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=49831" -Dfi
Student ID: 1001
Student ID: 1002

Process finished with exit code 0
```

The bottom status bar shows the file path 'World > src > UniqueID.java > UniqueID > main' and the time '16:22 to Ctrl+Shift+Enter'.

10 Eligibility.java

Abstract Class — Merit & Need-Based Scholarship Eligibility

The screenshot displays the IntelliJ IDEA IDE with the following components:

- Open Files:** A list of files in the 'Hello World' project, including 'src', 'AgePredictor', 'AttendancePercentage.java', 'BasicCalculator', 'Borrowing.java', and 'Courses.java'.
- Structure:** A tree view showing the class hierarchy: 'Scholarship' (with 'isEligible(double): boolean'), 'MeritScholarship' (with 'isEligible(double): boolean'), and 'NeedBaseScholarship' (with 'isEligible(double): boolean').
- Code Editor:** The 'Eligibility.java' file is open, showing the following code:

```
4 class MeritScholarship extends Scholarship{ 1 usage
7     return gpa >= 3.5;
8 }
9 }
10 class NeedBaseScholarship extends Scholarship { 1 usage
11     @Override 2 usages
12     boolean isEligible(double familyIncome){
13         return familyIncome < 30000;
14     }
15 }
16 public class Eligibility{
17     public static void main(String[] args){
18         Scholarship merit = new MeritScholarship();
19         Scholarship need = new NeedBaseScholarship();
20         System.out.println("Merit Scholarship: " + merit.isEligible(value: 3.5));
21         System.out.println("Need Base Scholarship: " + need.isEligible(value: 2500))
```
- Run:** The 'Eligibility' class is selected for execution.
- Output:** The console shows the execution results:

```
"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=49825" -Df
Merit Scholarship: true
Need Base Scholarship: true
Process finished with exit code 0
```
- Footer:** The bottom status bar shows the current file path: 'Hello World > src > Eligibility.java > Eligibility > main'.